Metric Logic Verification Report

DA3408 Assignment 4 · CPU image captioning monitored with Prometheus, node_exporter and Grafana

Name: S Samrudh Raaj, Roll No: DA24B025 · 25 October 2026

Summary of results

#	Metric	Hand calculation	Prometheus / dashboard	Result
M1	Throughput (Counter → rate)	0.6667 img/s	0.6667 img/s (rate) · 0.6667 img/s (panel)	✔ **match**
M2	Image error ratio (Counter → ratio)	3.06 %	3.06 % (lifetime)	✔ **match**
M2b	Error ratio of this run vs dashboard tile	4.00 %	4.00 % (tile, increase[5m])	✔ **match**
M3	Avg bulk request time (Summary _sum ÷ _count)	40.381 s	40.381 s (lifetime)	✔ **match**
M3b	This bulk job's duration vs app UI	33.823 s	33.8 s (app 'Total time')	✔ **match**
M4	p95 inference latency (Histogram)	1.8875 s	1.8875 s (histogram_quantile)	✔ **match**
M5	Host CPU busy (node Counter)	69.0 %	69.2 % (same 10 s window) · 62.1 % (panel, 30 s)	✔ **match**
M6	App share of host CPU (process Counter)	38.5 %	37.5 % (same 10 s window) · 32.4 % (panel, 30 s)	✔ **match**
M7	Host memory used (Gauges)	67.69 %	67.31 % (panel)	✔ **match**
M8	Disk free on / (Gauges)	48.49 %	48.49 % (panel)	✔ **match**

10 of 10 checks match within their tolerance. Lifetime counters, the Summary and the Histogram (M2, M3, M4) match exactly because both sides use the same scraped values. Rates and CPU shares (M1, M5, M6) match within a few percent when Prometheus uses the same window; the dashboard panels use 30 s windows for CPU, so during a ramp-up they read a few points lower than a 10 s calculation (explained under M5 and M6).

1. Purpose and method

The assignment's Black Box Metric Rule allows AI-generated PromQL but requires the metric logic to be calculated manually. For each metric below, raw values were read directly from the exporters (`/metrics` on port 18000 for the app and 9100 for node_exporter) or as raw samples from Prometheus. The metric's formula was then applied with plain arithmetic, without PromQL functions, and the result was compared with the value Prometheus and the Grafana panel report for the same moment.

Disclosure: at my request, the AI coding assistant captured the raw values during the recorded run and carried out the arithmetic shown in this report. The arithmetic is laid out step by step so that every number can be re-derived by hand during evaluation (see `AI_DISCLOSURE.md`).

The workload was one single image followed by one ZIP of 23 valid images, 1 truncated JPEG and 1 text file. Exporter snapshots were taken before any work (S0), after the single image (S1), twice during the bulk job (S2a, S2b, about 10 s apart) and after the bulk job finished (S3). Lifetime counters also include jobs the app had processed earlier; values marked this run subtract the S0 baseline.

2. Hand calculations

M1 · Throughput: images captioned per second (Counter → rate)

Metric: `caption_images_processed_total{mode="bulk",status="success",model="blip-base"}`. **Raw samples:** the last 20 s of scrapes ending at 22:54:40.998, read from Prometheus as a range vector (no functions applied).

Sample time	Unix time (s)	Counter value
22:54:23.760	1792949063.760	73
22:54:28.760	1792949068.760	76
22:54:33.760	1792949073.760	80
22:54:38.760	1792949078.760	83

Formula: rate = $(v_{last} - v_{first}) \div (t_{last} - t_{first})$
Working: $(83 - 73) \div (1792949078.760 - 1792949063.760) = 10 \div 15.000 =$ **0.6667 images/s**
✔ **match** Prometheus `rate(...[20s])` at the same instant = 0.6667; the Grafana Throughput panel (`sum by (mode) (rate(...[$__rate_interval]))`, 20 s) shows bulk = 0.6667. The values are identical because the counter rose at a steady rate, so extrapolating the slope to the full 20 s window does not change it.

M2 · Image error ratio (Counter → ratio)

Metric: `caption_images_processed_total{status=...}`, model blip-base, read from `:18000/metrics`.

Series	S0 (before)	S3 (after)
single/success	2	3
single/failed	0	0
bulk/success	69	92
bulk/failed	2	3

Formula: error ratio = failed ÷ (failed + success)
Lifetime (S3): 3 ÷ 98 = **3.061 %**. ✔ **match** Prometheus `sum(failed) / sum(all)` at S3 = 3.061 %.
This run (S3 − S0): 1 ÷ 25 = **4.000 %**. ✔ **match** The Grafana Image error ratio tile (`increase(...[5m])`, i.e. the dashboard's 5-minute range, which covers only this run) shows 4.000 %.

M3 · Average bulk request time (Summary _sum ÷ _count)

Metric: `caption_request_seconds_sum` and `caption_request_seconds_count` for mode bulk.

Series	S0	S3	S3 − S0
_sum (seconds)	87.321	121.144	33.823
_count (requests)	2	3	1

Formula: average = _sum ÷ _count
Lifetime: 121.144 ÷ 3 = **40.381 s**. ✔ **match** Prometheus `sum(_sum) / sum(_count)` = 40.381 s.
This bulk job: 33.823 ÷ 1 = **33.823 s**. ✔ **match** The app's own Total time for the job was 33.8 s, timed independently in app.py.

M4 · p95 inference latency per image (Histogram)

Metric: `caption_inference_seconds_bucket{mode="bulk",model="blip-base"}` at S3 (cumulative bucket counts).

le (s)	0.05	0.1	0.25	0.5	0.75	1	1.5	2	3	5	8	13	20	30	+Inf
count	0	0	0	0	0	5	75	91	92	92	92	92	92	92	92

Formula (linear interpolation inside the bucket, as histogram_quantile does): rank = 0.95 × N; find the first bucket whose count ≥ rank; p95 = lower + (upper − lower) × (rank − c_{lower}) ÷ (c_{upper} − c_{lower})
Working: N = 92, rank = 0.95 × 92 = 87.40. First bucket with count ≥ 87.40 is le = 2 (count 91); the bucket below is le = 1.5 (count 75). p95 = 1.5 + (2 − 1.5) × (87.40 − 75) ÷ (91 − 75) = **1.8875 s**
✔ **match** Prometheus `histogram_quantile(0.95, sum by (le) (...))` on the same counts = 1.8875 s.
Using only this run's bulk images (S3 − S1 per bucket) gives p95 = 1.9893 s (N = 23).

M5 · Host CPU busy (node_exporter Counter)

Metric: `node_cpu_seconds_total{mode="idle"}`, one counter per logical CPU, summed over CPUs and read twice during the bulk job.

	S2a	S2b
Wall-clock time	22:54:31.009	22:54:40.998
Σ idle seconds over all CPUs	145742.13	145810.16

Formula: CPU busy = 1 − Δidle ÷ (Δt × n_{cpu}), with n_{cpu} = 22 and Δt = 9.988 s
Working: 1 − (145810.16 − 145742.13) ÷ (9.988 × 22) = 1 − 68.03 ÷ 219.74 = **69.0 %**
✔ **match** Prometheus over the same 10 s window, `1 - avg(rate(node_cpu_seconds_total{mode="idle"}[10s]))` at S2b, gives 69.2 %. The CPU · host utilisation panel uses a 30 s window (recording rule `instance:node_cpu_utilisation:ratio`) and shows 62.1 % at that moment: its window still includes the seconds before the bulk job ramped up, so it lags the 10 s hand calculation.

M6 · Caption app's share of host CPU (process Counter)

Metric: `process_cpu_seconds_total` of the app process, read at S2a and S2b.

	S2a	S2b
process_cpu_seconds_total	972.07	1056.75

Formula: app share = Δcpu_seconds ÷ (Δt × n_{cpu})
Working: (1056.75 − 972.07) ÷ (9.988 × 22) = 84.68 ÷ 219.74 = **38.5 %**
✔ **match** Prometheus over the same 10 s window, `rate(process_cpu_seconds_total[10s]) / count(cpus)` at S2b, gives 37.5 %. The panel's caption app process line (`job:caption_process_cpu_share_of_host:ratio`, 30 s window) shows 32.4 % for the same reason as M5. In the hand calculation the app accounts for 56 % of all busy CPU, which is the app-vs-host correlation the dashboard is built to show.

M7 · Host memory used (Gauges)

Gauge (at S2b)	Bytes
node_memory_MemTotal_bytes	15,653,351,424
node_memory_MemAvailable_bytes	5,057,167,360

Formula: used = 1 − MemAvailable ÷ MemTotal = 1 − 5,057,167,360 ÷ 15,653,351,424 = **67.69 %**
✔ **match** The Memory · host utilisation panel shows 67.31 %.

M8 · Free space on the root filesystem (Gauges)

Gauge (at S3, mountpoint /)	Bytes
node_filesystem_avail_bytes	28,482,977,792
node_filesystem_size_bytes	58,735,820,800

Formula: free = avail ÷ size = 28,482,977,792 ÷ 58,735,820,800 = **48.49 %**
✔ **match** The Disk free space bar for / shows 48.49 %.